Web Scraping Tabanlı Kentsel Haber İzleme ve Harita Üzerinde Görselleştirme Sistemi

Farahnozkhon Dilovarovna MASUMOVA
Kocaeli Üniversitesi
Bilgisayar Mühendisliği Bölümü
Yazılım Laboratuvarı II – Proje I Raporu
2025–2026 Bahar Dönemi

Hayrunisa KORKULU
Kocaeli Üniversitesi
Bilgisayar Mühendisliği Bölümü
Yazılım Laboratuvarı II – Proje I Raporu
2025–2026 Bahar Dönemi

***Özet*—Bu projede, Kocaeli iline ait yerel haber kaynaklarından belirli haber türlerini otomatik olarak toplayan, işleyen ve harita üzerinde görselleştiren web tabanlı bir *kentsel haber izleme sistemi* geliştirilmiştir. Sistem; web scraping modülü ile yerel haber sitelerinden trafik kazası, yangın, elektrik kesintisi, hırsızlık ve kültürel etkinlikler gibi olaylara ilişkin haberleri toplamaktadır. Toplanan veriler ön işleme aşamasından geçirilerek HTML etiketleri, gereksiz boşluklar, özel karakterler ve alakasız içerikler temizlenmiş; ardından haber türü otomatik olarak sınıflandırılmıştır.**

Aynı haberin birden fazla kez kaydedilmesini önlemek amacıyla metin benzerliği analizi uygulanmış; içerik olarak büyük ölçüde benzer haberler tek kayıt altında birleştirilmiştir. Haber metinlerinden olayın geçtiği konum bilgisi çıkarılmış, bu bilgiler geocoding servisi kullanılarak enlem-boylam koordinatlarına dönüştürülmüş ve MongoDB veritabanında saklanmıştır. Elde edilen haber verileri, Google Maps tabanlı bir kullanıcı arayüzü üzerinde marker'lar ile gösterilmiş; kullanıcıların haber türü, ilçe ve tarih bilgisine göre dinamik filtreleme yapabilmesi sağlanmıştır.

Geliştirilen sistem, dağınık halde bulunan yerel haber verilerini tek bir platformda toplayarak kullanıcıya hızlı, görsel ve anlamlı bir izleme deneyimi sunmaktadır. Bu yönüyle çalışma; veri toplama, metin işleme, konum çıkarımı, veri görselleştirme ve kullanıcı etkileşimi bileşenlerini bir araya getiren bütünleşik bir şehir bilgi sistemi örneği oluşturmaktadır.

Anahtar Kelimeler — Web Scraping, MongoDB, Google Maps, Haber Sınıflandırma, Konum Çıkarımı, Geocoding, Veri Görselleştirme, Kentsel Haber İzleme.

I. Giriş

Günümüzde şehir yaşamı; trafik kazaları, yangınlar, elektrik kesintileri, hırsızlık olayları ve kültürel etkinlikler gibi çok sayıda dinamik gelişmeden etkilenmektedir. Bu tür olaylara ilişkin bilgiler çoğunlukla yerel haber siteleri üzerinden kamuoyuna duyurulmaktadır. Ancak bu içeriklerin farklı kaynaklarda dağınık halde bulunması, kullanıcıların güncel olaylara hızlı ve bütüncül biçimde erişmesini zorlaştırmaktadır. Özellikle aynı olayın birden fazla haber kaynağında farklı başlıklarla yer alması, konum bilgisinin açık biçimde verilmemesi ve içeriklerin manuel olarak takip edilmesinin zaman alması, şehir ölçeğinde çalışan bir haber izleme sistemine olan ihtiyacı artırmaktadır.

Bu projede, Kocaeli iline ait yerel haber kaynaklarından veri toplayan ve bu verileri harita üzerinde görselleştiren web tabanlı bir **kentsel haber izleme sistemi** geliştirilmiştir. Sistem, belirlenen yerel haber sitelerini otomatik olarak tarayarak haber başlığı, haber içeriği, yayın tarihi, haber kaynağı ve haber bağlantısı gibi temel bilgileri elde etmektedir. Ardından bu haberler; **Trafik Kazası, Yangın, Elektrik Kesintisi, Hırsızlık** ve **Kültürel Etkinlikler** olmak üzere proje isterlerinde verilen kategorilere otomatik olarak sınıflandırılmaktadır. Bu sayede kullanıcı, çok sayıdaki yerel haber içeriğini tek tek incelemek zorunda kalmadan olay türüne göre filtrelenmiş ve anlamlandırılmış bilgilere erişebilmektedir.

Projede yalnızca veri çekme işlemi gerçekleştirilmemiş; scraping sonrasında elde edilen metinler üzerinde çeşitli ön işleme adımları da uygulanmıştır. HTML etiketlerinin temizlenmesi, fazla boşlukların giderilmesi, gereksiz özel karakterlerin ayıklanması ve metnin normalize edilmesi gibi işlemlerle haber içerikleri analiz edilebilir hale getirilmiştir. Ayrıca aynı haberin birden fazla sitede tekrar etmesi problemi de göz önünde bulundurulmuş; hem haber bağlantısı bazında hem de içerik benzerliği üzerinden tekrar kontrolü yapılmıştır. Özellikle farklı kaynak-

larda yayımlanan benzer haberlerin tek bir kayıt altında birleştirilmesi amacıyla *embedding* tabanlı metin benzerliği yaklaşımı kullanılmıştır. Böylece kullanıcı arayüzünde aynı olayın gereksiz biçimde çok sayıda tekrar gösterilmesi engellenmiştir.

Sistemin önemli bileşenlerinden biri de **konum çıkarımı ve koordinat dönüşümü** sürecidir. Haber metinlerinden ilçe, mahalle, sokak, cadde veya belirli mekân isimleri tespit edilmeye çalışılmış; elde edilen konum ifadesi daha sonra geocoding servisi kullanılarak enlem-boylam bilgisine dönüştürülmüştür. Geocoding işlemlerinde tekrar eden sorguların önüne geçmek amacıyla bir önbellek (*cache*) yapısı kullanılmış, böylece aynı konum için gereksiz API çağrıları azaltılmıştır. Konum bilgisi başarıyla çıkarılan ve koordinata dönüştürülen haberler, harita üzerinde gösterilmek üzere MongoDB veritabanında saklanmıştır.

Geliştirilen sistem istemci–sunucu mimarisine göre tasarlanmıştır. Backend katmanında Python ve Flask kullanılmış; veri saklama işlemleri MongoDB üzerinde gerçekleştirilmiştir. Scraping işlemleri modüler scraper sınıfları ile yürütülmüş; farklı haber siteleri için ayrı veri toplama mantıkları tanımlanmıştır. Kullanıcı arayüzü tarafında ise filtreleme paneli, haber istatistik kartları, son haber listesi ve Google Maps tabanlı harita bileşeni bir araya getirilmiştir. Harita üzerinde her haber, türüne göre farklı renk ve ikon ile işaretlenmekte; kullanıcı marker'a tıkladığında haber başlığı, tarih, konum ve kaynak bağlantılarını içeren ayrıntılı bilgi penceresi açılmaktadır. Ayrıca tarih aralığı, olay türü ve ilçe filtresi sayesinde harita yeniden yüklenmeden dinamik filtreleme yapılabilmektedir.

Bu rapor kapsamında; geliştirilen sistemin gereksinimleri ve kapsamı, scraping mimarisi, veritabanı yapısı, haber sınıflandırma yaklaşımı, konum çıkarımı ve geocoding süreci, kullanıcı arayüzü tasarımı ve harita görselleştirme bileşenleri ayrıntılı olarak sunulacaktır. Bunun yanında tekrar eden haberlerin birleştirilmesi, Kocaeli dışı içeriklerin elenmesi ve filtreleme mekanizmalarının sistem davranışına etkisi değerlendirilerek genel bir tartışma yapılacaktır.

II. Yöntem ve Sistem Mimarisi

Bu bölümde geliştirilen kentsel haber izleme sisteminin yazılım mimarisi, veri işleme akışı, veritabanı yapısı, scraping bileşenleri, haber türü sınıflandırma yaklaşımı, konum çıkarımı ve harita görselleştirme katmanı açıklanmaktadır. Sistem istemci–sunucu mimarisine göre geliştirilmiştir. Veri toplama, ön işleme, sınıflandırma ve koordinat üretimi gibi iş mantıkları backend tarafında; kullanıcı etkileşimi ve harita sunumu ise frontend tarafında gerçekleştirilmiştir. Bu yapı sayesinde sistem modüler, geliştirilebilir ve bakımı kolay bir hale getirilmiştir.

A. Genel Mimari

Sistem üç temel katmandan oluşmaktadır:

- **Veri Toplama ve İşleme Katmanı:** Kocaeli'ye ait yerel haber sitelerinden veri çekmek için scraper modülleri geliştirilmiştir. Her haber kaynağı için ayrı bir sınıf tanımlanmış; böylece sitelerin HTML yapıları birbirinden bağımsız biçimde işlenebilmiştir.
- **Backend Katmanı (Flask + MongoDB):** Flask tabanlı sunucu uygulaması; haberlerin listelenmesi, filtrelenmesi, ilçe bilgilerinin döndürülmesi, scraping işleminin yeniden tetiklenmesi ve istatistiklerin sağlanması gibi görevleri yerine getirmektedir. Veri saklama için MongoDB kullanılmıştır.
- **Frontend Katmanı (HTML/CSS/JavaScript + Google Maps):** Kullanıcı arayüzü; filtre paneli, kategori sayaçları, son haber listesi ve harita bileşeninden oluşmaktadır. Haberler Google Maps üzerinde kategoriye göre farklı renk ve ikonlarla gösterilmektedir.

Bu ayrım sayesinde scraping, veritabanı işlemleri, haber analizi ve kullanıcı arayüzü birbirinden bağımsız geliştirilebilmiş; bakım ve hata ayıklama süreçleri kolaylaştırılmıştır.

B. Backend Mimarisi ve API Yapısı

Backend tarafında Flask kullanılmış ve istemci ile sunucu arasındaki iletişim JSON tabanlı API uç noktaları üzerinden sağlanmıştır. Uygulamada ayrıca `flask-cors` kullanılarak istemci tarafının sunucuya güvenli biçimde erişebilmesi sağlanmıştır. Veri erişimi MongoDB koleksiyonları üzerinden yürütülmüştür.

Sistemde kullanılan temel API uç noktaları aşağıdaki gibidir:

- `/`: Ana sayfanın ve harita arayüzünün sunulması

- `/api/haberler`: Filtrelenmiş haber listesinin getirilmesi
- `/api/istatistik`: Tür bazlı haber istatistiklerinin döndürülmesi
- `/api/scrape`: Scraping işleminin yeniden başlatılması
- `/api/ilceler`: Veritabanındaki ilçe bilgilerinin liste halinde döndürülmesi

`/api/haberler` uç noktasında yalnızca geçerli haber türüne sahip, Kocaeli içi olduğu doğrulanmış ve koordinat bilgisi bulunan haberler döndürülmektedir. Ayrıca olay türü, ilçe ve tarih aralığı parametrelerine göre filtreleme yapılabilmektedir. Bu yapı, kullanıcı arayüzündeki dinamik filtreleme mekanizmasının temelini oluşturmaktadır.

C. Veritabanı Tasarımı

Projede veritabanı olarak MongoDB kullanılmıştır. Bu tercih, haber içeriklerinin yarı-yapısal doğasına uygun esnek bir veri modeli sunmaktadır. Sistemde temel olarak iki koleksiyon bulunmaktadır:

- **`haberler`**: Haber başlığı, içerik, yayın tarihi, haber türü, kaynak adı, haber bağlantısı, konum metni, ilçe bilgisi, enlem-boylam değerleri, kaynak listesi ve ek bağlantılar gibi alanları tutar.
- **`geocache`**: Daha önce koordinata dönüştürülmüş konum ifadelerini saklayarak aynı adres için tekrar geocoding çağrısı yapılmasını engeller.

Performans ve veri bütünlüğü amacıyla `haber_linki`, `haber_turu`, `ilce`, `yayin_tarihi` ve `konum_metni` alanları üzerinde indeksler tanımlanmıştır. Özellikle `haber_linki` için benzersiz indeks kullanılması, aynı haberin aynı bağlantı ile birden fazla kez kaydedilmesini engellemektedir.

D. Web Scraping Modülü

Sistem, proje isterlerinde verilen beş farklı Kocaeli yerel haber kaynağını tarayacak şekilde tasarlanmıştır: Çağdaş Kocaeli, Özgür Kocaeli, Ses Kocaeli, Yeni Kocaeli ve Bizim Yaka. Her kaynak için `BaseScraper` sınıfından türetilmiş ayrı scraper sınıfları yazılmıştır. Bu sınıflar, ilgili sitenin ana sayfasından haber bağlantılarını toplamakta, ardından her haberin detay sayfasına girerek başlık, içerik, yayın tarihi ve bağlantı bilgisini çıkarmaktadır.

`BaseScraper` sınıfı ortak scraping davranışlarını merkezi hale getirmektedir. Bu sınıfta:

- kullanıcı ajanı (*user-agent*) döndürme,
- istekler arasında bekleme ekleme,
- HTML ve özel karakter temizleme,
- tarih normalizasyonu,
- haberin son 3 gün içinde olup olmadığını kontrol etme,
- ilçe ve konum bilgisi çıkarma,
- haberin Kocaeli içi olup olmadığını belirleme

gibi ortak işlemler gerçekleştirilmiştir. Böylece tüm scraper'larda tekrar eden kod azaltılmıştır.

E. Veri Temizleme ve Ön İşleme

Scraping sonrasında elde edilen metinler doğrudan kullanılmamış; analiz öncesinde çeşitli ön işleme adımlarından geçirilmiştir. Bu süreçte HTML etiketleri silinmiş, fazla boşluklar temizlenmiş, gereksiz özel karakterler ayıklanmış ve metin Unicode normalizasyonuna tabi tutulmuştur. Bazı haber sitelerinde yer alan reklam, yorum, telif ve bilgilendirme metinleri de kara liste yaklaşımıyla ayıklanmıştır. Bu yaklaşım, özellikle haber içeriğinin sınıflandırma aşamasında daha temiz ve anlamlı olmasını sağlamıştır.

F. Haber Türü Sınıflandırma Yaklaşımı

Projede haber türünü belirlemek için iki aşamalı bir sınıflandırma yaklaşımı kullanılmıştır. İlk aşamada kural tabanlı sınıflandırma uygulanmıştır. Bu aşamada haber başlığı ve içerik metni birlikte değerlendirilmiş; kategoriye özgü anahtar kelimeler, destekleyici ifadeler ve dışlayıcı kelimeler kullanılmıştır. Her kategori için eşleşen kelimelere göre puan hesaplanmış; puan belirli bir eşik değerin üzerindeyse haber ilgili türe atanmıştır. Bir haberin birden fazla kategoriye yakın görünmesi durumunda ise önceden tanımlanan öncelik sırası ve bağlam kontrolü uygulanmıştır.

Kural tabanlı yöntem yeterli güvenle sonuç vermezse ikinci aşama devreye alınmıştır. Bu aşamada `sentence-transformers` tabanlı çok dilli embedding modeli kullanılmıştır. Haber metninin vektör temsili oluşturulmuş; bu temsil, her kategori için hazırlanmış örnek metin vektörleri ile karşılaştırılmıştır. En yüksek benzerlik skoruna sahip kategori, eşik ve ayrım koşulları sağlandığında nihai sınıf olarak kabul edilmiştir. Böylece sistem, hem

anahtar kelime tabanlı temel isterleri karşılamakta hem de daha esnek anlamsal sınıflandırma desteği sunmaktadır.

G. Kullanılan Anahtar Kelimeler ve Öncelik Kuralları

Projede haber türü sınıflandırması iki aşamalı olarak gerçekleştirilmiştir. İlk aşamada kural tabanlı sınıflandırma uygulanmıştır. Bu aşamada her kategori için güçlü, orta, zayıf ve kara liste ifadeleri tanımlanmıştır. Eşleşmeler başlık ve içerik üzerinde ayrı ayrı sayılmış; başlıktaki eşleşmelere daha yüksek ağırlık verilmiştir. Daha sonra toplam puan hesaplanmış ve ön koşul denetimi ile birlikte en uygun kategori seçilmiştir. Kod yapısında kategori öncelik sırası şu şekilde tanımlanmıştır: `Trafik Kazası`, `Yangın`, `Elektrik Kesintisi`, `Hırsızlık`, `Kültürel Etkinlikler`.

Bu çalışmada kullanılan anahtar kelimeler ve kurallar aşağıdaki gibidir.

Trafik Kazası: Güçlü ifadeler arasında "trafik kazası", "zincirleme kaza", "trafik kazasında", "motosiklet kazası", "otobüs kazası", "ölümlü kaza", "maddi hasarlı kaza", "kavşakta kaza", "feci kaza", "iki araç çarpıştı", "otomobil ile motosiklet çarpıştı", "tır ile otomobil çarpıştı", "kamyondan düşen tomruk", "otomobile saplandı", "direksiyon hakimiyetini kaybetti", "kontrolden çıktı", "yayaya çarptı" ve "yol ulaşıma kapandı" yer almaktadır. Orta düzey ifadeler arasında "çarpıştı", "takla attı", "şarampole", "kaza meydana", "tır devrildi", "araç takla", "kazada yaralandı", "bariyere çarptı", "refüje çarptı", "kamyon devrildi" ve "otomobil devrildi" bulunmaktadır. Kara listede ise "iş kazası", "depremzede", "siyasi kaza", "kazanan", "turnuva", "yarışma", "müsabaka", "sabiha gökçen hattı", "yolcular yolda kaldı" ve "kriz" ifadeleri yer almaktadır.

Yangın: Güçlü ifadeler arasında "yangın çıktı", "çıkan yangın", "ev yangını", "fabrika yangını", "çatı yangını", "orman yangını", "otluk yangını", "baca yangını", "yangına müdahale", "itfaiye ekipleri müdahale", "yangın söndürüldü", "alevlere teslim oldu", "alev alev yandı", "yangın kontrol altına alındı", "soğutma çalışması yapıldı", "olay yerine itfaiye ekipleri sevk edildi", "şofben alev aldı" ve "araç alev aldı" bulunmaktadır. Orta düzey ifadeler "ev yandı", "fabrika yandı", "araç alev aldı", "yangın paniği" ve "bacasında yangın" biçimindedir. Kara listede "davasında", "davasının", "davası", "yargılandığı", "sanık", "sanıklar", "tutuklu", "tahliye", "mahkeme", "duruşma", "savunma", "müşteki", "ceza infaz", "savcı", "saldırı", "tabancayla", "ateş açıldı", "silahlı", "uyuşturucu", "narkotik", "operasyonu", "baskın yapılan", "zabıta", "denetim", "ağaçlandırma", "fidan" ve "destek" yer almaktadır.

Elektrik Kesintisi: Güçlü ifadeler arasında "elektrik kesintisi", "elektrikler kesilecek", "planlı kesinti", "elektrik verilmeyecek", "elektrik kesilecek", "elektrik kesileceği", "enerji kesintisi", "elektriksiz kalacak" ve "elektrik kesinti programı" vardır. Orta düzey ifadeler "SEDAŞ", "trafo arızası", "bakım çalışması" ve "enerji verilemeyecek" biçimindedir. Zayıf düzey ifadeler ise "kesinti" ve "trafo" olarak tanımlanmıştır. Kara listede "elektrik faturası", "elektrik akımı" ve "elektrikli araç" yer almaktadır.

Hırsızlık: Güçlü ifadeler arasında "hırsızlık", "hırsız yakalandı", "hırsız tutuklandı", "hırsızlık şüphelisi", "ev soyuldu", "araç çalındı", "kapkaç", "gasp", "soygun", "kablo hırsızlığı", "kablo vurgunu", "çalıntı motosiklet" ve "çalıntı araç" bulunmaktadır. Orta düzey ifadeler arasında "kasayı çaldı", "yankesici" ve "çalıntı çıktı" vardır. Kara listede ise "fikri mülkiyet", "şarkıyı çaldı", "karakol", "polis öğrenci buluşması", "öğrenci buluşması", "dolandırıldı", "dolandırıcılık", "elektrik akımı", "kaza", "kazaya", "otomobil devrildi", "tır dorsesi", "yola dökülen atıklar" ve "zincirleme kaza" yer almaktadır.

Kültürel Etkinlikler: Güçlü ifadeler arasında "konser düzenlendi", "konser verildi", "tiyatro oyunu", "tiyatro gösterimi", "sergi açıldı", "sergi düzenlendi", "festival düzenlendi", "müzik festivali", "resim sergisi", "kitap fuarı", "kültürel etkinlik", "söyleşi düzenlendi" ve "bilim söyleşisi" bulunmaktadır. Orta düzey ifadeler "sahne aldı", "sahneye çıktı", "dans gösterisi", "müzik dinletisi", "kutlama töreni", "etkinlik düzenlendi", "panel düzenlendi", "sanat sergisi" ve "kültür merkezi etkinlik" biçimindedir. Zayıf düzey ifadeler ise "konser", "festival", "sergi" ve "tiyatro" olarak belirlenmiştir. Kara listede "maç", "gol", "penaltı", "lig", "spor", "futbol", "basketbol", "belediyespor", "deplasman", "galatasaray", "fenerbahçe", "beşiktaş", "şampiyonluk", "turnuva", "yarı final", "final", "milli takım", "antrenör", "karşılaşma", "skor", "puan", "belediye

başkanı", "meclis", "operasyon", "rüşvet", "istifa", "seçim", "parti", "milletvekili", "vali", "kentsel dönüşüm", "altyapı", "çöp kamyonu", "ihale", "proje sunumu", "protokol", "hastası", "tedavi", "ameliyat", "sma", "uyuşturucu", "narkotik", "gözaltı", "saldırı", "cinayet", "ateş açıldı", "tabancayla", "ziraat odası", "ticaret odası", "arıcılara", "fındık fiyat", "kamera", "cctv", "zabıta", "denetim" ve "baskın yapılan" gibi ifadeler vardır.

Sınıflandırma yalnızca anahtar kelime geçişine göre yapılmamıştır. Kodda ayrıca kategoriye özgü ön koşul denetimi bulunmaktadır. Örneğin Trafik Kazası kategorisinde "kaza" ifadesinin yanında araç veya yol bağlamı da aranmıştır. Yangın kategorisinde itfaiye veya yangın bağlamı beklenmiştir. Kültürel Etkinlikler kategorisinde ise spor, siyaset, mahkeme, asayiş ve benzeri dışlayıcı bağlamlar özellikle elenmiştir.

Kural tabanlı sonuç yeterli olmazsa ikinci aşamada semantic fallback kullanılmıştır. Bu aşamada haber metni için embedding oluşturulmuş, kategori örnek cümlelerinden elde edilen vektörlerle benzerlik hesaplanmıştır. En iyi skorun en az `0.45` olması ve ilk iki skor arasındaki farkın en az `0.05` olması koşulları uygulanmıştır. Son olarak semantic sonuç da yine kategori ön koşulu ile doğrulanmıştır.

H. Tekrar Eden Haberlerin Birleştirilmesi

Aynı haberin farklı yerel kaynaklarda yayınlanması durumuna karşı embedding tabanlı benzerlik analizi uygulanmıştır. Haber başlığı ile içerik özetinden oluşturulan metinler için embedding vektörleri üretilmiş; farklı sitelerden gelen haber çiftleri arasında kosinüs benzerliği hesaplanmıştır. Benzerlik oranı belirlenen eşik değerinin üzerine çıktığında haberler aynı olay olarak kabul edilmiştir. Bu durumda ana haber kaydında `kaynak_listesi` ve `ek_linkler` alanları güncellenmiş, tekrarlayan kayıt ise silinmiştir. Böylece harita üzerinde aynı olayın çok sayıda kopyası yerine, tek işaretçi altında birden fazla kaynak gösterilebilmiştir.

I. Konum Çıkarımı ve Geocoding

Proje isterlerinde belirtildiği üzere, konum çıkarım yöntemi raporda ayrıntılı olarak açıklanmıştır. Bu çalışmada konum çıkarımı, haber metni içerisindeki yer belirten ifadelerin kural tabanlı desenlerle tespit edilmesi üzerine kurulmuştur. İlk aşamada metin içinde Kocaeli ilçeleri aranmıştır. Ardından mahalle, sokak, cadde, bulvar, yol adı ve belirli mekân isimleri tespit edilmeye çalışılmıştır. Böylece mümkün olan en spesifik konum bilgisinin elde edilmesi hedeflenmiştir.

Konum çıkarımı sırasında şu mantık izlenmiştir: önce mahalle + ilçe biçimindeki ifadeler aranmış, bunlar bulunamazsa sokak, cadde veya bulvar bilgileri değerlendirilmiş, daha sonra TEM, D-100 ve D-130 gibi yol adları kontrol edilmiştir. Eğer bunlar da yoksa kültür merkezi, park, müze, salon ve benzeri belirli mekân adları dikkate alınmıştır. Hiçbiri bulunamazsa son aşamada yalnızca ilçe düzeyinde konum atanmıştır. Bu yaklaşım, proje isterindeki "adres bilgisi net değilse mümkün olan en spesifik konum alınabilir" şartıyla uyumludur.

Ayrıca sistem yalnızca Kocaeli ili sınırları içindeki haberleri işlemek üzere tasarlanmıştır. Bu nedenle metinde yalnızca başka şehir adları bulunuyor ve Kocaeli bağlamı oluşmuyorsa ilgili haber sistem dışı bırakılmıştır. Konum bulunamayan haberler ise haritada gösterilmemiştir. Bu davranış da proje isterlerinde açıkça belirtilen kurallarla uyumludur.

Elde edilen konum metni daha sonra koordinat üretimi için geocoding aşamasına aktarılmıştır. Bu aşamada önce `geocache` koleksiyonu kontrol edilmiştir. Aynı konum daha önce işlenmişse doğrudan kayıtlı koordinatlar kullanılmıştır. Önbellekte bulunmayan konumlar için Google Geocoding API çağrısı yapılmıştır. Eğer konum yalnızca ilçe düzeyindeyse, önceden tanımlanmış ilçe merkez koordinatları yaklaşık çözüm olarak kullanılabilmiştir. Geocoding işlemi başarısız olursa haber kaydı harita üzerinde gösterilmemiştir. Böylece hem gereksiz API çağrıları azaltılmış hem de kullanıcıya yalnızca doğrulanmış konuma sahip haberler sunulmuştur.

J. Frontend ve Harita Görselleştirme

Frontend tarafında tek sayfalık bir web arayüzü geliştirilmiştir. Arayüzün sol panelinde tarih aralığı, olay türü ve ilçe filtreleri; tür bazlı sayaç kartları; renk açıklamaları ve son haberler listesi yer almaktadır. Harita bölümünde ise Google Maps kullanılarak haberler marker biçiminde gösterilmektedir. Her haber türü için ayrı renk ve emoji tabanlı görsel gösterim tercih edilmiştir.

Bir marker'a tıklandığında açılan bilgi penceresinde haber türü, başlık, tarih, konum bilgisi ve kay-

nak bağlantıları gösterilmektedir. Eğer aynı haber birden fazla kaynaktan toplanmışsa, kullanıcıya bu kaynakların her biri için ayrı bağlantı sunulmaktadır. Ayrıca filtreleme işlemleri istemci tarafından parametreli API çağrıları ile gerçekleştirilmekte; böylece harita yeniden yüklenmeden dinamik güncelleme sağlanmaktadır.

K. Veri Akışı Özeti

Sistemin genel veri akışı aşağıdaki gibi özetlenebilir:

1) Yerel haber siteleri scraper modülleri ile taranır.
2) Elde edilen haberler temizlenir, tarihleri normalize edilir ve Kocaeli içi kontrolünden geçirilir.
3) Haber türü otomatik olarak belirlenir.
4) Metinden konum bilgisi çıkarılır ve geocoding ile koordinat üretilir.
5) Haber kayıtları MongoDB'ye kaydedilir.
6) Embedding tabanlı benzerlik analizi ile tekrar eden haberler birleştirilir.
7) Kullanıcı arayüzü, API üzerinden filtrelenmiş haberleri çekerek harita üzerinde gösterir.

Bu mimari sayesinde sistem; veri toplama, metin analizi, konum çıkarımı ve harita tabanlı görselleştirme işlevlerini uçtan uca yerine getirebilen bütünleşik bir yazılım çözümü sunmaktadır.

III. Algoritma ve Yalancı Kod

Bu bölümde sistemin çekirdek veri toplama, haber işleme, sınıflandırma, konum çıkarımı, koordinat üretimi ve görselleştirme akışı yalancı kod biçiminde sunulmaktadır. Amaç, sistemin hangi adımlarla çalıştığını açıkça göstermek ve gerçek kod ile kavramsal akış arasındaki ilişkiyi netleştirmektir.

A. Algoritma ve Yalancı Kod Ayrımı

Algoritma, bir problemi çözmek için izlenen mantıksal adımların tanımıdır. Örneğin haberleri çekmek, temizlemek, sınıflandırmak, konum çıkarmak ve haritada göstermek bir algoritmik akış oluşturur.

Yalancı kod ise bu akışı gerçek programa benzer biçimde, ancak dile bağımlı olmadan anlatan gösterimdir. Bu raporda verilen algoritmalar, gerçek uygulamadaki işleyişi özetleyen yalancı kod mantığıyla hazırlanmıştır.

B. Problem Tanımı

Sistem; Kocaeli yerel haber sitelerinden son günlere ait haberleri otomatik olarak toplar, haber metinlerini temizler, bunları belirlenen haber türlerinden birine atar, metinden konum bilgisi çıkarır, bu bilgiyi koordinata dönüştürür ve sonucu harita üzerinde kullanıcıya sunar.

Desteklenen zorunlu haber türleri şunlardır:

- `Trafik Kazası`
- `Yangın`
- `Elektrik Kesintisi`
- `Hırsızlık`
- `Kültürel Etkinlikler`

Sistem yalnızca Kocaeli ili ve ilçelerine ait haberleri işlemeyi hedefler. Kocaeli dışı haberler ve belirlenen türler dışında kalan kayıtlar veritabanına alınmaz veya daha sonra temizlenir.

C. Çekirdek Akış: Scraping Sürecini Başlatma

Aşağıdaki akış, sistemdeki "tüm haberleri güncelle" davranışını özetlemektedir. Bu işlem hem bağımsız betik ile hem de backend tarafındaki `/api/scrape` uç noktası ile tetiklenebilir.

Algoritma 1: Tüm Kaynakları Tara ve İşle

1) Scraper listesi oluştur:
   - `CagdasKocaeliScraper`
   - `OzgurKocaeliScraper`
   - `SesKocaeliScraper`
   - `YeniKocaeliScraper`
   - `BizimYakaScraper`
2) Veritabanındaki mevcut haber sayısını kaydet.
3) Her scraper için sırayla:
   - `scrape()` metodunu çalıştır.
   - Hata olursa logla ve diğer scraper'a devam et.
4) Tüm scraping tamamlandıktan sonra:
   - koordinatsız haberler için geocoding çalıştır,
   - ardından embedding tabanlı benzerlik analizi çalıştır.
5) Yeni toplam haber sayısını hesapla.
6) Çıktı olarak toplam haber sayısı, yeni eklenen kayıt sayısı ve işlem durumu mesajı üret.

D. Site Bazlı Scraping Mantığı

Her haber sitesi için benzer ama kaynağa özgü bir scraping akışı vardır. Önce ana sayfa indirilir,

sonra uygun bağlantılar seçilir, ardından haber detay sayfaları işlenir.

Algoritma 2: Bir Haber Kaynağını Tara

1) Girdi: `site_adi`, `ana_url`.
2) Ana sayfayı indir.
3) Sayfa indirilemezse işlemi sonlandır.
4) HTML içindeki bağlantıları gez.
5) Belirli desenlere uyan haber linklerini seç.
6) Her uygun bağlantı için haber detayını çek.
7) Haber geçerliyse ve yeni tarihliyse veritabanına kaydet.
8) Çıktı olarak bulunan ve kaydedilen haber sayısını üret.

E. Haber Detay Sayfasını İşleme

Her haber sayfasında başlık, içerik ve yayın tarihi çıkarılır. Daha sonra sınıflandırma ve konum bulma işlemleri uygulanır.

Algoritma 3: Haber Detayını Çek ve Yapılandır

1) Girdi: haber detay bağlantısı `url`.
2) Sayfayı indir.
3) Başlık alanını bul.
4) Başlık boşsa veya çok kısa ise haberi geçersiz say.
5) İçerik alanını bul.
6) Gerekirse siteye özgü haber bloğunu kullan.
7) Bulunamazsa anlamlı paragrafları topla.
8) İçeriği temizle:
   - HTML etiketlerini kaldır,
   - fazla boşlukları temizle,
   - özel karakterleri normalize et.
9) İçerik çok kısa ise haberi geçersiz say.
10) Yayın tarihini çıkar.
11) Haber türünü belirle.
12) Tür belirlenemiyorsa haberi alma.
13) Konum metnini bul.
14) Haber kaydını oluştur.
15) Çıktı: haber nesnesi veya `None`.

F. Tarih Çıkarma ve Normalizasyon

Haberlerin filtrelenebilmesi için tarih alanı ortak bir biçime dönüştürülür.

Algoritma 4: Tarihi Çıkar

1) Girdi: `soup`, ham HTML.
2) Önce `application/ld+json` bloklarında `datePublished` alanını ara.
3) Bulunamazsa `time datetime` alanını kontrol et.
4) Hâlâ bulunamazsa uygun `meta` etiketlerini ara.
5) Son çare olarak metin içinde tarih desenlerini tara.
6) Hiçbiri bulunamazsa "Tarih Bulunamadı" döndür.

Algoritma 5: Tarihi ISO Biçimine Dönüştür

1) Girdi: `tarih_metni`.
2) Tarih zaten `YYYY-MM-DD` biçimindeyse olduğu gibi kullan.
3) Tarih "GG Ay YYYY" biçimindeyse ay adını sayıya çevir.
4) Tarih "GG.AA.YYYY" biçimindeyse parçalayarak dönüştür.
5) Dönüştürülemiyorsa `None` döndür.
6) Çıktı: normalize edilmiş ISO tarih.

G. Haber Türü Sınıflandırma

Sistemde iki aşamalı sınıflandırma vardır. Önce kural tabanlı yaklaşım uygulanır. Sonuç alınamazsa semantic fallback devreye girer.

Algoritma 6: Haber Türünü Belirle

1) Girdi: `baslik`, `icerik`.
2) Önce kural tabanlı sınıflandırmayı çalıştır.
3) Sonuç bulunduysa bu sonucu kullan.
4) Sonuç yoksa semantic sınıflandırmayı çalıştır.
5) Haber metni için embedding üret.
6) Kategori örnek embedding'leri ile benzerlik hesapla.
7) En yüksek skorlu kategoriyi seç.
8) Skor yeterince yüksek değilse sonuç üretme.
9) İlk iki skor çok yakınsa sonucu belirsiz kabul et.
10) Semantic sonuç bulunsa bile kategori ön koşulunu yeniden doğrula.
11) Doğrulanırsa sonucu döndür; aksi halde `None` döndür.

Algoritma 7: Kural Tabanlı Kategori Skoru Hesapla

1) Her kategori için:
   - güçlü ifadeleri başlık ve içerikte ara,
   - orta ifadeleri ara,
   - zayıf ifadeleri ara,
   - kara liste ifadelerini ara.
2) Her eşleşme için kategori skorunu güncelle.
3) Güçlü eşleşme yüksek puan, orta eşleşme orta puan, zayıf eşleşme düşük puan getirir.

4) Kara liste eşleşmesi puanı düşürür.
5) En yüksek skorlu kategorileri büyükten küçüğe sırala.
6) Her kategori için bağlam ve ön koşul kontrolü yap.
7) Eşik üstünde ve ön koşulu sağlayan ilk kategoriyi sonuç olarak döndür.
8) Hiçbiri uygun değilse `None` döndür.

H. Kocaeli İçeriğini Doğrulama ve Konum Çıkarımı

Projede sadece Kocaeli ili sınırları içindeki haberler işlenir. Bu yüzden haberin Kocaeli ile ilişkili olup olmadığı ayrıca kontrol edilir.

Algoritma 8: Konum Metni Bul

1) Girdi: haber metni.
2) Metin boşsa "Belirtilmemiş" döndür.
3) Metindeki Kocaeli ilçelerini ara.
4) Yalnızca başka şehir isimleri varsa ve Kocaeli bağı yoksa "Belirtilmemiş" döndür.
5) Öncelik sırasına göre desen eşleştir:
   - mahalle bilgisi,
   - sokak, cadde veya bulvar bilgisi,
   - yol adı,
   - kültür merkezi, park, müze, salon gibi mekân adı,
   - yalnızca ilçe bilgisi.
6) Mahalle + ilçe bulunduysa bu biçimde döndür.
7) Yol + ilçe bulunduysa bu biçimde döndür.
8) Yalnızca ilçe bulunduysa ilçe düzeyinde döndür.
9) Hiçbir uygun ifade yoksa "Belirtilmemiş" döndür.

Algoritma 9: Haber Kocaeli İçinde mi?

1) Girdi: `baslik`, `icerik`.
2) Metinde Kocaeli ilçelerinden biri geçiyor mu kontrol et.
3) "Kocaeli" kelimesi geçiyor mu kontrol et.
4) Geniş bir diğer şehir ve ilçe kara listesi üzerinde arama yap.
5) Kocaeli ilçesi veya açık "Kocaeli" ifadesi varsa `True`.
6) Yalnızca başka şehirler varsa `False`.
7) Belirsiz durumda `False`.

I. Veritabanına Kaydetme

Geçersiz türler, Kocaeli dışı haberler ve tekrar eden linkler veritabanına alınmaz.

Algoritma 10: Haberi Veritabanına Kaydet

1) Girdi: haber nesnesi.
2) Haber türü boşsa veya "Diğer" ise kaydetme.
3) Başlık ve içerik üzerinden haberin Kocaeli içi olup olmadığını kontrol et.
4) Kocaeli dışı ise kaydetme.
5) Aynı `haber_linki` veritabanında varsa kaydetme.
6) Yayın tarihini normalize et.
7) İlçe bilgisini önce `konum_metni`'nden bulmaya çalış.
8) Gerekirse başlık ve içerikten çıkar.
9) `ek_linkler` alanı yoksa boş liste oluştur.
10) Haberi `haberler` koleksiyonuna ekle.
11) Çıktı: başarı durumu.

J. Geocoding Süreci

Konum metni bulunan ama koordinatı olmayan haberler için koordinat üretimi yapılır.

Algoritma 11: Koordinat Bul

1) Girdi: `konum_metni`.
2) Konum boşsa veya "Belirtilmemiş" ise başarısız dön.
3) Konum metnini normalize et.
4) Önce geocache içinde ara.
5) Önbellekte varsa doğrudan koordinatları döndür.
6) Metin sadece ilçe ise ön tanımlı ilçe koordinatlarını kullan.
7) Değilse Google Geocoding API çağrısı yap.
8) API sonucu başarılıysa cache'e yaz ve döndür.
9) API başarısızsa son kez ilçe eşleştirmesi ile yaklaşık koordinat dene.
10) Yine olmazsa `None, None` döndür.

Algoritma 12: Google Geocoding Çağrısı

1) Girdi: normalize edilmiş konum metni.
2) API anahtarı yoksa işlemi sonlandır.
3) Adreste "Kocaeli" geçmiyorsa sonuna "Kocaeli, Türkiye" ekle.
4) Google Geocoding API'ye istek gönder.
5) Sonuçlar geldiyse her sonucu Kocaeli içinde mi diye kontrol et.
6) Uygun ilk sonucu seç.
7) Koordinatları döndür.
8) Uygun sonuç yoksa başarısız dön.

Algoritma 13: Tüm Koordinatsız Haberleri Geocode Et

1) Veritabanından koordinatı olmayan uygun haberleri çek.
2) Her haber için `koordinat_bul(konum_metni)` çalıştır.
3) Başarılıysa enlem ve boylam alanlarını güncelle.
4) Başarısızsa geocode durumunu başarısız olarak işaretle.
5) İşlem sonunda başarılı ve başarısız sayaçlarını raporla.

K. Benzer Haberleri Birleştirme

Aynı olayın farklı kaynaklarda tekrar etmesini engellemek için embedding tabanlı benzerlik analizi yapılır.

Algoritma 14: Haberler Arası Benzerlik Analizi

1) Veritabanındaki haberleri çek.
2) Her haber için temsil metni oluştur.
3) Tüm metinler için embedding üret.
4) Her haber çifti için benzerliği hesapla.
5) Aynı kaynaktan gelen haberleri atla.
6) Benzerlik eşiğin üzerindeyse ilk haberi ana kayıt kabul et.
7) Diğer kaynağı `kaynak_listesi` alanına ekle.
8) Ek bağlantıyı `ek_linkler` alanına yaz.
9) Tekrarlayan haberi sil.
10) Çıktı olarak birleştirilen kayıt sayısını üret.

L. API Akışları

Frontend, veriyi Flask API üzerinden çeker ve haritayı buna göre günceller.

Algoritma 15: Filtrelenmiş Haberleri Getir

1) Varsayılan filtreyi oluştur.
2) Koordinatı olan ve geçerli haber türüne sahip kayıtları seç.
3) Gerekirse tür filtresini uygula.
4) Gerekirse ilçe filtresini uygula.
5) Gerekirse tarih aralığını uygula.
6) Sonuçları yayın tarihine göre azalan sırala.
7) JSON olarak döndür.

Algoritma 16: İstatistikleri Getir

1) Geçerli haberler için ortak filtreyi oluştur.
2) Toplam haber sayısını hesapla.
3) Koordinatı olan haber sayısını hesapla.
4) Her kategori için kayıt sayısını hesapla.
5) Sonucu JSON olarak döndür.

Algoritma 17: İlçeleri Listele

1) Koordinatı olan ve geçerli haberlerden farklı ilçe değerlerini çek.
2) Boş olanları çıkar.
3) İlçeleri alfabetik sırala.
4) JSON liste olarak döndür.

M. Frontend Harita ve Filtreleme Akışı

Kullanıcı tarafında harita ve sol panel birlikte çalışır.

Algoritma 18: Haritayı Başlat

1) Google Maps nesnesini Kocaeli merkez koordinatları ile oluştur.
2) Varsayılan tarih aralığını son 3 gün olacak şekilde ayarla.
3) Haberleri yükle.
4) İlçe listesini yükle.

Algoritma 19: Harita Üzerine Marker Ekle

1) Girdi: bir haber kaydı.
2) Haber türüne göre renk ve emoji seç.
3) Üst üste binmeyi azaltmak için küçük ofset uygula.
4) Marker oluştur.
5) Popup içeriğini hazırla.
6) Marker'a tıklanınca bilgi penceresini aç.

Algoritma 20: Filtre Uygula

1) Sol panelden olay türü, ilçe ve tarih bilgisini al.
2) Seçili değerlerden sorgu parametreleri oluştur.
3) `/api/haberler` çağrısı yap.
4) Dönen haberleri haritada yeniden göster.
5) Son haberler listesini güncelle.
6) İstatistik sayaçlarını güncelle.

N. Hata ve Dayanıklılık Akışları

Sistem, scraping ve API katmanında oluşabilecek hatalara karşı temel dayanıklılık mekanizmalarına sahiptir.

Algoritma 21: Sayfa İndirme Hatasını Yönet

1) Haber sayfasını indirirken HTTP hatası oluşursa logla.
2) Hata kodu 429 ise bekle ve isteği bir kez daha dene.
3) İkinci deneme de başarısız olursa `None` döndür.
4) Diğer scraper'ların çalışmasını engelleme.

Algoritma 22: Sistem Kontrolü

1) `.env` dosyası var mı kontrol et.
2) Google Maps API anahtarı tanımlı mı kontrol et.
3) Google Geocoding API anahtarı tanımlı mı kontrol et.
4) MongoDB bağlantısını test et.
5) Gerekli Python kütüphanelerinin yüklü olup olmadığını kontrol et.
6) Hataları ve eksikleri kullanıcıya raporla.

O. Veri Akışı Özeti

Sistemin genel veri akışı aşağıdaki gibi özetlenebilir:

1) Yerel haber siteleri taranır.
2) Başlık, içerik ve tarih bilgileri çıkarılır.
3) Metin temizlenir ve normalize edilir.
4) Haber türü otomatik olarak belirlenir.
5) Kocaeli içi kontrolü yapılır.
6) Konum metni çıkarılır.
7) Koordinat üretimi yapılır.
8) Haber veritabanına kaydedilir.
9) Benzer haberler birleştirilir.
10) API üzerinden frontend'e sunulur.
11) Kullanıcı filtreleyerek harita üzerinde haberleri görüntüler.

IV. Deneysel Sonuçlar

Bu bölümde geliştirilen web scraping tabanlı kentsel haber izleme sisteminin gerçek uygulama ekranları ve sistem akışı üzerinden elde edilen sonuçları değerlendirilmektedir. Kargo planlama projesinden farklı olarak bu çalışmada sabit sayısal senaryolardan çok, sistemin haber toplama, sınıflandırma, konum çıkarımı, harita üzerinde gösterim ve filtreleme yetenekleri test edilmiştir. Değerlendirme; hem backend tarafındaki kod akışı hem de kullanıcı arayüzüne ait ekran görüntüleri üzerinden yapılmıştır. Sistem; yerel haber kaynaklarından veri çekme, haberleri uygun kategoriye atama, aynı olaya ait tekrar eden kayıtları birleştirme ve bunları harita üzerinde anlamlı biçimde sunma açısından uçtan uca çalışır durumda gözlemlenmiştir.

A. Deney Kurulumu ve Test Ortamı

Deneyler, Python tabanlı Flask backend, MongoDB veritabanı ve Google Maps destekli web arayüzü üzerinde gerçekleştirilmiştir. Sistemde haber toplama işlemi, kodda tanımlanan beş ayrı scraper sınıfı üzerinden yürütülmektedir: `Çağdaş Kocaeli`, `Özgür Kocaeli`, `Ses Kocaeli`, `Yeni Kocaeli` ve `Bizim Yaka`. Scraping işlemi tamamlandıktan sonra koordinatı eksik haberler için geocoding, ardından da tekrar eden içeriklerin birleştirilmesi amacıyla embedding tabanlı benzerlik analizi uygulanmaktadır. Bu işlem akışı, hem bağımsız çalıştırma betiğiyle hem de kullanıcı arayüzündeki "Haberleri Güncelle" düğmesi ile tetiklenebilmektedir.

Sistem arayüzünde ayrıca olay türü, ilçe ve tarih aralığına göre filtreleme yapılabilmekte; filtreleme sonucunda yalnızca uygun haber kayıtları harita üzerinde yeniden gösterilmektedir. Bu yapı, sistemin sadece veri toplamakla kalmadığını, aynı zamanda kullanıcıya etkileşimli bir analiz ortamı sunduğunu göstermektedir.

B. Arayüz Üzerinden Gözlenen Sonuçlar

Elde edilen ekran görüntülerine göre sistemin ana sayfasında sol tarafta bir kontrol paneli, sağ tarafta ise harita bileşeni yer almaktadır. Kontrol panelinde başlangıç ve bitiş tarihi seçilebilmekte, olay türü filtresi (*Trafik Kazası, Yangın, Elektrik Kesintisi, Hırsızlık, Kültürel Etkinlikler*) ve ilçe filtresi uygulanabilmektedir. Ayrıca her haber türü için sayaç kartları gösterilmekte ve kullanıcıya mevcut veri dağılımı hakkında hızlı bir özet sunulmaktadır. Bu yapı, ödev isterlerinde belirtilen haber türü, ilçe ve tarih seçimine göre filtreleme gereksinimini doğrudan karşılamaktadır.

Harita üzerinde her haber, türüne göre farklı renk ve simgelerle gösterilmektedir. Bu tasarım sayesinde kullanıcı, tek bakışta olay tiplerini ayırt edebilmektedir. Örneğin trafik kazaları kırmızı tonları ve araç simgesiyle, yangın haberleri turuncu tonları ve ateş simgesiyle, elektrik kesintileri sarı tonları ve yıldırım simgesiyle gösterilmektedir. Kültürel etkinlikler ve hırsızlık haberleri için de benzer biçimde ayrı görsel kodlar kullanılmıştır. Kod tarafında bu yapı `TUR_RENKLERI` ve `TUR_EMOJILERI` sözlükleri ile tanımlanmıştır.

Marker'a tıklandığında açılan bilgi penceresinde haber türü, başlık, tarih, konum bilgisi ve kaynak bağlantıları gösterilmektedir. Aynı haber birden fazla sitede bulunduğunda kullanıcıya yalnızca tek bir marker gösterilmekte, ancak bilgi penceresinde ilgili kaynakların tümü ayrı bağlantılar halinde su-

nulmaktadır. Bu davranış, hem kullanıcı arayüzünde veri tekrarını azaltmakta hem de farklı kaynaklardan doğrulama yapabilme imkânı sağlamaktadır. Ekran görüntülerinde bu davranış açık biçimde görülmektedir; bir haber seçildiğinde popup içerisinde birden fazla "Habere Git" bağlantısı listelenebilmektedir. Bu sonuç, embedding tabanlı haber birleştirme mekanizmasının arayüz düzeyinde başarıyla yansıtıldığını göstermektedir.

C. Filtreleme Mekanizmasının Değerlendirilmesi

Gönderdiğin ekran görüntülerinde hem ilçe filtresi hem de olay türü filtresi uygulanmış örnekler yer almaktadır. İlçe bazlı filtre uygulandığında harita üzerinde yalnızca seçilen ilçeye ait marker'ların kaldığı; olay türü filtresinde ise yalnızca ilgili kategoriye ait haberlerin gösterildiği görülmektedir. Bu da frontend tarafında filtre parametrelerinin doğru şekilde `/api/haberler` uç noktasına gönderildiğini ve backend tarafında uygun MongoDB sorgularının oluşturulduğunu göstermektedir.

Tarih aralığı filtresi için de kullanıcı arayüzünde iki ayrı tarih alanı bulunmaktadır. Kod yapısında başlangıç ve bitiş tarihleri isteğe bağlı parametreler olarak alınmakta; bu bilgiler mevcutsa sorguya `$gte` ve `$lte` koşulları eklenmektedir. Dolayısıyla sistem yalnızca tür ve ilçe bazında değil, zamansal olarak da filtrelenebilen bir haber izleme aracı haline gelmiştir.

D. Scraping ve Veri Güncelleme Sürecinin Değerlendirilmesi

Kullanıcı arayüzünde yer alan "Haberleri Güncelle" düğmesi, sistemin en önemli işlevlerinden birini oluşturmaktadır. Bu düğme aracılığıyla backend tarafında tüm scraper sınıfları tekrar çalıştırılmakta, ardından geocoding ve benzerlik analizi adımları yürütülmektedir. Böylece sistem, yalnızca başlangıçta veri yükleyen statik bir uygulama değil, istendiğinde güncel veri ile yeniden beslenebilen dinamik bir yapı sunmaktadır. Projede istenen "scraping işlemi sistem çalıştığında tekrar tetiklenebilir olmalıdır" şartı bu şekilde sağlanmıştır.

Ayrıca kodda HTTP hataları, özellikle de yoğun istek sonucu oluşabilecek `429 Too Many Requests` durumları için temel bir dayanıklılık mekanizması da bulunmaktadır. İstek başarısız olduğunda belirli bir süre beklenip yeniden deneme yapılması, gerçek ortam koşullarında scraper modüllerinin daha dayanıklı çalışmasına katkı sağlamaktadır.

E. Konum Çıkarımı ve Geocoding Sonuçlarının Değerlendirilmesi

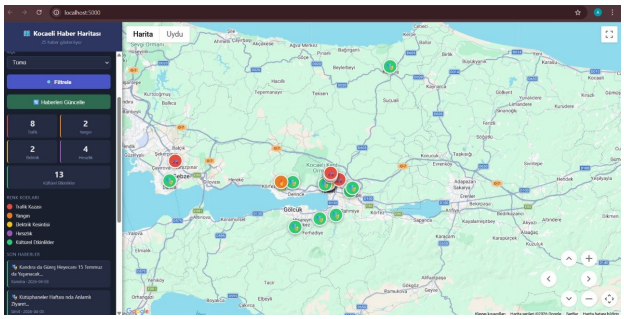
Sistem, haber metinlerinden doğrudan koordinat üretmek yerine önce konum metni çıkarmakta, ardından bu metni geocoding işlemine sokmaktadır. Kod yapısında ilçe, mahalle, sokak, cadde, bulvar ve belirli mekân kalıpları aranmakta; mümkün olan en spesifik konum elde edilmeye çalışılmaktadır. Geocoding tarafında ise önce önbellek kontrolü yapılmakta, ardından Google Geocoding API kullanılarak koordinat bilgisi üretilmektedir. Sadece ilçe adı bulunan durumlarda ise önceden tanımlanmış ilçe koordinatları yaklaşık çözüm olarak kullanılabilmektedir. Bu yaklaşım, hem API çağrısı sayısını azaltmakta hem de konum bulunamayan haber sayısını düşürmektedir.

Harita üzerinde yalnızca koordinatı başarıyla üretilmiş haberlerin gösterilmesi, sistemin veri kalitesini koruma açısından doğru bir tercih olduğunu göstermektedir. Ödev isterlerinde de konum bulunamayan haberlerin haritada gösterilmemesi gerektiği özellikle belirtilmiştir; geliştirilen sistem bu kuralla uyumludur.

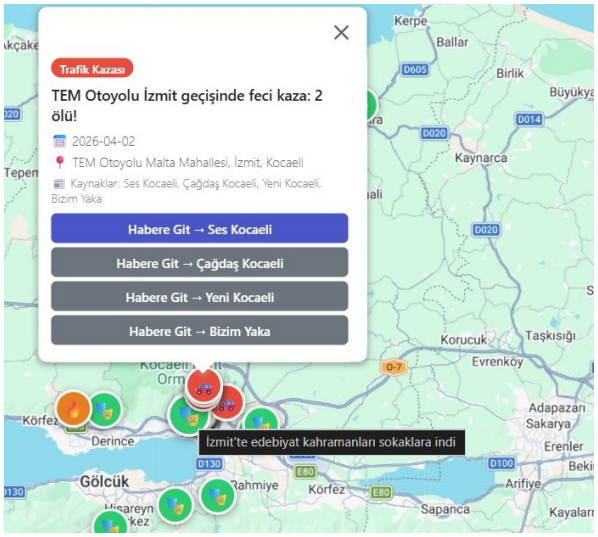
F. Nitel Sonuçlar ve Genel Yorum

Genel olarak değerlendirildiğinde sistem, proje kapsamında istenen temel işlevleri başarıyla yerine getirmektedir. Yerel Kocaeli haber kaynaklarından veri toplanabilmekte, haberler beş zorunlu kategoriye ayrılabilmekte, konum bilgileri çıkarılıp koordinata dönüştürülebilmekte ve sonuçlar harita üzerinde filtrelenebilir biçimde sunulabilmektedir. Ayrıca aynı olayın farklı kaynaklarda tekrar etmesi problemi benzerlik analizi ile azaltılmıştır.

Elde edilen sonuçlar, geliştirilen uygulamanın gerçek haber verileriyle çalışan, etkileşimli ve geliştirilebilir bir şehir olay izleme sistemi olduğunu göstermektedir. Özellikle filtreleme başarısı, popup bilgi pencereleri, istatistik kartları ve çoklu kaynak desteği, sistemin kullanıcı deneyimi açısından yeterli bir düzeye ulaştığını ortaya koymaktadır. Bu yönüyle sistem, hem teknik gereksinimleri karşılayan hem de pratik kullanım sunan bir uygulama niteliği taşımaktadır.



Şekil 1: Sistemin genel harita arayüzü.



Şekil 2: Haber detay bilgi penceresi.

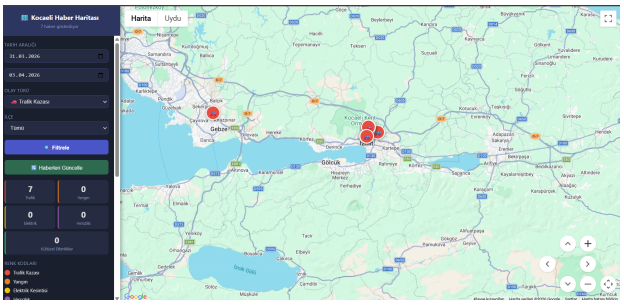
G. Grafiksel ve Görsel Değerlendirme

Yüklenen ekran görüntüleri, sistemin arayüz çıktılarının değerlendirilmesi açısından önemli görsel kanıtlar sunmaktadır. Şekil 1'da sistemin genel harita görünümü, Şekil 2'da bir haber popup'ına ait ayrıntılar, Şekil 3'de ilçe filtresi uygulanmış görünüm ve Şekil 4'de olay türü filtresi uygulanmış görünüm yer almaktadır. Bu görseller, sistemin hem işlevsel hem de görsel açıdan başarılı biçimde tamamlandığını göstermektedir.

V. Sonuç

Bu projede, Kocaeli iline ait yerel haber kaynaklarından veri toplayan, bu verileri işleyen ve harita üzerinde görselleştiren web tabanlı bir kentsel haber izleme sistemi başarıyla geliştirilmiştir. Geliştirilen sistem; farklı yerel haber sitelerinden otomatik veri çekebilmesi, haberleri belirlenen türlere göre sınıflandırabilmesi, metinlerden konum bilgisi

Şekil 3: İlçe filtresi uygulanmış harita görünümü.



Şekil 4: Olay türü filtresi uygulanmış harita görünümü.

çıkarabilmesi ve elde edilen olayları harita üzerinde kullanıcıya sunabilmesi gibi temel gereksinimleri kapsamaktadır.

Sistem mimarisi, Python ve Flask kullanılarak geliştirilen modüler bir backend yapısı üzerine kurulmuş; MongoDB veritabanı ile desteklenmiştir. Frontend tarafında ise HTML, CSS ve JavaScript tabanlı kullanıcı arayüzü ile Google Maps entegrasyonu kullanılarak sade, işlevsel ve etkileşimli bir kullanım deneyimi sağlanmıştır. Bu mimari yapı sayesinde sistem; veri toplama, veri saklama, veri filtreleme ve görselleştirme süreçlerini uçtan uca yürütebilen bütünleşik bir uygulama haline gelmiştir.

Projede haber toplama işlemi için her haber kaynağına özgü scraper sınıfları geliştirilmiştir. Bu yapı sayesinde sitelerin farklı HTML düzenleri ayrı ayrı ele alınabilmiş ve ortak işlemler `BaseScraper` sınıfı altında birleştirilmiştir. Scraping sonrasında elde edilen metinler temizlenmiş, normalize edilmiş ve analiz için uygun hale getirilmiştir. Böylece ham ve dağınık haber içerikleri, işlenebilir ve sınıflandırılabilir veri yapılarına dönüştürülmüştür.

Haber türü sınıflandırma sürecinde iki aşamalı bir yaklaşım benimsenmiştir. Öncelikle anahtar kelime ve kural tabanlı sınıflandırma uygulanmış, gerekli

durumlarda ise **embedding tabanlı anlamsal benzerlik** yaklaşımı yardımıyla semantic fallback kullanılmıştır. Bu sayede haberler; `Trafik Kazası`, `Yangın`, `Elektrik Kesintisi`, `Hırsızlık` ve `Kültürel Etkinlikler` kategorilerine daha tutarlı biçimde ayrılabilmiştir. Ayrıca aynı olayın farklı kaynaklarda tekrar eden haberler şeklinde görünmesi problemi, benzerlik analizi ile azaltılmış ve kullanıcıya daha temiz bir çıktı sunulmuştur.

Konum çıkarımı ve geocoding süreci, sistemin en önemli bileşenlerinden birini oluşturmaktadır. Haber metinlerinden ilçe, mahalle, sokak, cadde ve belirli mekân bilgileri çıkarılmaya çalışılmış; elde edilen metinler Google Geocoding API yardımıyla enlem-boylam bilgisine dönüştürülmüştür. Ayrıca tekrar eden konumlar için önbellek kullanılarak gereksiz API çağrılarının önüne geçilmiştir. Bu yaklaşım sayesinde hem performans artırılmış hem de konum bilgisi bulunan haberlerin harita üzerinde doğru biçimde gösterilmesi sağlanmıştır.

Kullanıcı arayüzü tarafında elde edilen sonuçlar, sistemin yalnızca teknik olarak çalıştığını değil, aynı zamanda kullanıcıya anlamlı ve etkileşimli bir deneyim sunduğunu göstermektedir. Harita üzerinde haberlerin türlerine göre farklı renk ve sembollerle gösterilmesi, popup pencerelerinde haber ayrıntılarının ve kaynak bağlantılarının sunulması, tarih aralığı, ilçe ve olay türüne göre filtreleme yapılabilmesi; projenin ödev isterleriyle uyumlu biçimde tamamlandığını göstermektedir. Özellikle aynı habere ait birden fazla kaynağın tek popup penceresi içinde listelenebilmesi, sistemin veri tekrarını azaltma konusundaki başarısını da yansıtmaktadır.

Sonuç olarak geliştirilen sistem;

- Kocaeli yerel haber kaynaklarından otomatik veri toplayabilen,
- Haberleri sınıflandırıp filtreleyebilen,
- Konum bilgisi çıkararak harita üzerinde gösterebilen,
- Çoklu kaynakları tek bir olay altında birleştirebilen,
- Etkileşimli ve kullanıcı dostu bir arayüz sunan

bir kentsel haber izleme altyapısı ortaya koymaktadır. Bu yönüyle çalışma, hem ders kapsamında istenen teknik gereksinimleri karşılamakta hem de şehir ölçeğinde olay takibi ve yerel bilgi sistemleri açısından geliştirilebilir bir temel sunmaktadır.

Gelecek çalışmalar kapsamında;

- Haber sınıflandırma başarımını artırmak için daha gelişmiş doğal dil işleme ve derin öğrenme tabanlı modellerin kullanılması,
- Konum çıkarımı aşamasında daha ayrıntılı adres çözümleme yöntemlerinin eklenmesi,
- Harita arayüzüne kümeleme (*clustering*), ısı haritası ve zaman çizelgesi gibi gelişmiş görselleştirme bileşenlerinin eklenmesi,
- Kullanıcılara gerçek zamanlı bildirim sağlamak amacıyla sistemin periyodik veya olay tetiklemeli otomatik scraping yapısına dönüştürülmesi,
- Yönetim ve analiz amaçlı ek istatistik panellerinin geliştirilmesi

sistemin başarımını, kullanılabilirliğini ve gerçek hayata uyarlanabilirliğini daha da artırabilecek geliştirmeler olarak önerilmektedir.

Kaynaklar

[1] Pallets, *Flask Documentation*. Çevrimiçi: https://flask.palletsprojects.com/

[2] MongoDB, Inc., *MongoDB Documentation*. Çevrimiçi: https://www.mongodb.com/docs/

[3] MongoDB, Inc., *PyMongo Documentation*. Çevrimiçi: https://pymongo.readthedocs.io/

[4] L. Richardson, *Beautiful Soup Documentation*. Çevrimiçi: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[5] K. Reitz et al., *Requests: HTTP for Humans Documentation*. Çevrimiçi: https://requests.readthedocs.io/

[6] Google for Developers, *Maps JavaScript API Documentation*. Çevrimiçi: https://developers.google.com/maps/documentation/javascript

[7] Google for Developers, *Geocoding API Documentation*. Çevrimiçi: https://developers.google.com/maps/documentation/geocoding

[8] Google for Developers, *Geocoding API: Requests and Responses*. Çevrimiçi: https://developers.google.com/maps/documentation/geocoding/guides-v3/requests-geocoding

[9] Google for Developers, *Maps JavaScript API — Markers*. Çevrimiçi: https://developers.google.com/maps/documentation/javascript/markers

[10] Google for Developers, *Maps JavaScript API — Info Windows*. Çevrimiçi: https://developers.google.com/maps/documentation/javascript/infowindows

[11] Sentence Transformers, *Sentence Transformers Documentation*. Çevrimiçi: https://www.sbert.net/

[12] Hugging Face, *sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 Model Card*. Çevrimiçi: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

[13] scikit-learn Developers, *cosine_similarity — scikit-learn Documentation*. Çevrimiçi: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

[14] NumPy Developers, *NumPy Documentation*. Çevrimiçi: https://numpy.org/doc/

[15] Python Software Foundation, *Python Documentation*. Çevrimiçi: https://www.python.org/doc/
[16] MDN Web Docs, *HTML: HyperText Markup Language*. Çevrimiçi: https://developer.mozilla.org/en-US/docs/Web/HTML
[17] MDN Web Docs, *CSS: Cascading Style Sheets*. Çevrimiçi: https://developer.mozilla.org/en-US/docs/Web/CSS
[18] MDN Web Docs, *JavaScript*. Çevrimiçi: https://developer.mozilla.org/en-US/docs/Web/JavaScript
[19] MDN Web Docs, *REST*. Çevrimiçi: https://developer.mozilla.org/en-US/docs/Glossary/REST
[20] MDN Web Docs, *HTTP Request Methods*. Çevrimiçi: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods
[21] MongoDB, Inc., *Single Field Indexes — MongoDB Manual*. Çevrimiçi: https://www.mongodb.com/docs/manual/core/indexes/index-types/index-single/
[22] MongoDB, Inc., *What Is NoSQL? NoSQL Databases Explained*. Çevrimiçi: https://www.mongodb.com/resources/basics/databases/nosql-explained
[23] K. Reitz, *Requests: HTTP for Humans*. Çevrimiçi: https://requests.readthedocs.io/en/latest/
[24] IEEE, *IEEE Conference Template and Guidelines*.
[25] Google for Developers, *Google Maps Platform Documentation*. Çevrimiçi: https://developers.google.com/maps